

Universidad del Valle de Guatemala

Facultad de Ingeniería

Laboratorio 8

Mini-GPT y embeddings contextuales

CC3092 - Deep Learning

Milton Polanco

Olivier Viau

4 de octubre de 2026

Índice

Resumen	1
1. Task 1 - Mini-GPT y muestreo	1
1.1. Task 1.1 - Dataset, vocabulario y batches	1
1.2. Task 1.2 - Entrenamiento y perplexidad	1
1.3. Task 1.3 - Estrategias de muestreo	3
2. Task 2 - Generación y análisis	3
2.1. Task 2.1 - Configuraciones	3
2.2. Task 2.2 - Discusión	4
3. Task 3 - Embeddings contextuales	6
3.1. Task 3.1 - Extracción con BERT	6
3.2. Task 3.2 - Similitud coseno y Word2Vec	6
3.3. Reducción PCA	7
4. Task 4 - Discusión conceptual	7
4.1. Task 4.1(a) - Necesidad de la máscara causal	7
4.2. Task 4.1(b) - Elección entre GPT y BERT	8
5. Conclusiones	8
Fuentes y herramientas	8

Resumen

Se entrenó un modelo autoregresivo Mini-GPT a nivel de carácter sobre el conjunto *tiny_shakespeare*. El modelo obtuvo una perplejidad de validación de 4.737, dentro del intervalo esperado de 3.5 a 8.0. Se compararon muestreo greedy, top-k y top-p bajo distintas temperaturas, observando el compromiso entre coherencia y diversidad. Posteriormente se extrajeron embeddings contextuales de la última capa de BERT multilingüe para tres palabras polisémicas en español. Las similitudes y la proyección PCA muestran que BERT modifica la representación de una palabra según su contexto, mientras que Word2Vec conserva un único vector por palabra.

1. Task 1 - Mini-GPT y muestreo

1.1. Task 1.1 - Dataset, vocabulario y batches

Se descargó *tiny_shakespeare*, compuesto por 1,115,394 caracteres. El vocabulario se construyó con los caracteres únicos del texto y contiene exactamente 65 elementos. Los diccionarios `stoi` e `itos` implementan los mapeos bidireccionales entre caracteres e índices enteros.

El texto completo se codificó como una secuencia de enteros y se dividió sin mezclar el orden temporal:

Partición	Número de caracteres
Entrenamiento	1,003,854
Validación	111,540

Cada batch tiene forma 64×128 . Si una entrada es $x = (x_1, \dots, x_{128})$, el objetivo correspondiente es $y = (x_2, \dots, x_{129})$. De esta forma, cada posición aprende a predecir el siguiente carácter.

Los hiperparámetros utilizados fueron:

Parámetro	Valor	Parámetro	Valor
<code>block_size</code>	128	<code>batch_size</code>	64
<code>d_model</code>	128	<code>lr</code>	3×10^{-4}
<code>n_heads</code>	4	Épocas	10
<code>n_layers</code>	3	Dropout	0.1

1.2. Task 1.2 - Entrenamiento y perplejidad

El modelo contiene embeddings de token y posición, tres bloques Transformer, cuatro cabezas de atención por bloque, capas de normalización y un MLP de dimensión interna $4d_{model}$. En total tiene aproximadamente 0.63 millones de parámetros. La atención causal se calcula como

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V, \quad (1)$$

donde M asigna $-\infty$ a las posiciones futuras. La función de pérdida es cross-entropy sobre los 65 caracteres del vocabulario.

Cuadro 1: Pérdida al final de cada época.

Época	Train	Validación	Perplejidad
1	2.2427	2.2566	9.55
2	1.8685	1.9494	7.02
3	1.6733	1.8107	6.11
4	1.5748	1.7363	5.68
5	1.5178	1.6715	5.32
6	1.4703	1.6321	5.11
7	1.4461	1.6083	4.99
8	1.4183	1.5895	4.90
9	1.3942	1.5676	4.80
10	1.3784	1.5555	4.74

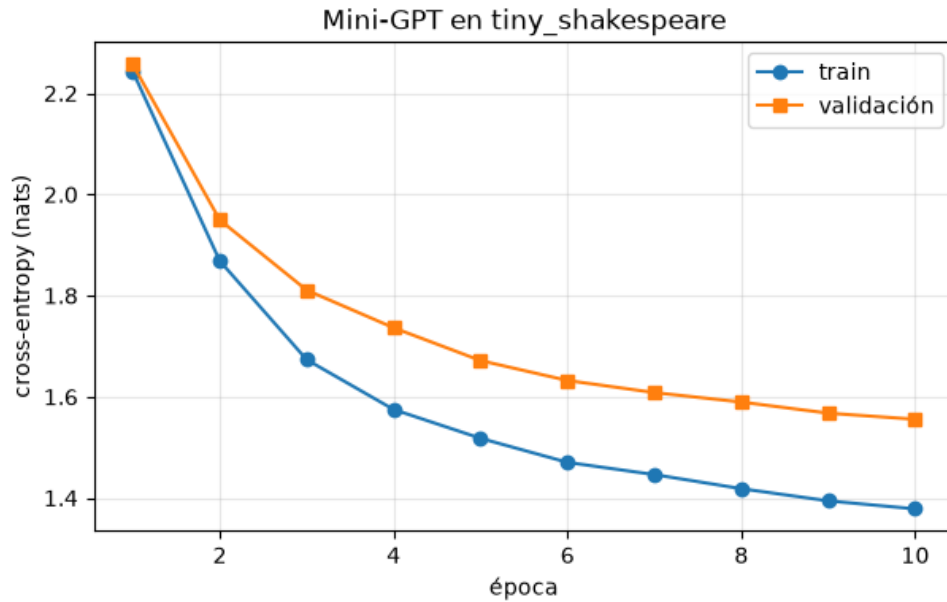


Figura 1: Pérdida de entrenamiento y validación por época.

La evaluación cubrió los 111,539 tokens predecibles de validación, incluida la última ventana incompleta. La perplejidad final fue

$$\text{PPL} = \exp \left(\frac{1}{N} \sum_{t=1}^N \mathcal{L}_t \right) = \exp(1.5555) = 4.737. \quad (2)$$

El valor está dentro del rango esperado y la Figura 1 muestra una reducción estable de ambas

pérdidas. La separación moderada entre entrenamiento y validación indica un ajuste razonable sin divergencia durante las diez épocas.

1.3. Task 1.3 - Estrategias de muestreo

Greedy

Se elige siempre el índice de mayor logit:

$$x_{t+1} = \arg \max_i z_i. \quad (3)$$

No hay aleatoriedad, por lo que un mismo contexto siempre produce la misma continuación.

Top-k con temperatura

Los logits se dividen por τ , se conservan exactamente los k mayores y el resto se reemplaza por $-\infty$. Después se aplica softmax y se toma una muestra de la distribución resultante:

$$P(w_i \mid ctx) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}. \quad (4)$$

Top-p con temperatura

Las probabilidades se ordenan de mayor a menor y se conserva el conjunto mínimo cuya suma acumulada alcanza p . El conjunto se renormaliza antes de muestrear. A diferencia de top-k, el número de candidatos cambia con la incertidumbre del modelo.

Las pruebas unitarias confirmaron que greedy devuelve el máximo, top-k con $k = 2$ solo produce los dos índices de mayor logit y top-p con $p = 0.5$ conserva únicamente el primer índice para el vector de prueba utilizado.

2. Task 2 - Generación y análisis

2.1. Task 2.1 - Configuraciones

Se generaron cinco muestras de 200 caracteres nuevos para cada configuración, siempre a partir del prompt `ROMEO:`. Las 25 salidas completas se conservan en el notebook entregado; aquí se presentan las métricas y los fragmentos necesarios para sustentar el análisis.

Cuadro 2: Configuraciones de generación.

Configuración	Estrategia	Parámetros
A	Greedy	Ninguno
B	Top-k	$k = 5, \tau = 0.7$
C	Top-k	$k = 50, \tau = 1.0$
D	Top-p	$p = 0.70, \tau = 0.7$
E	Top-p	$p = 0.95, \tau = 1.0$

Como medida auxiliar se calculó la proporción de trigramas distintos sobre los 200 caracteres generados:

Cuadro 3: Diversidad de las muestras.

Configuración	Muestras únicas	Distinct-3
A	1/5	0.278
B	5/5	0.716
C	5/5	0.883
D	5/5	0.741
E	5/5	0.880

2.2. Task 2.2 - Discusión

a. Repetición en greedy

Las cinco muestras A son idénticas porque la regla $\arg \max$ es determinista. En todas aparece la secuencia “The prince of the prince”, seguida por nuevas repeticiones de “the prince” y “the state”. Cuando el modelo vuelve a un contexto similar, toma las mismas decisiones y refuerza el patrón. Esto explica que A tenga una sola muestra única y el menor valor de distinct-3.

El fenómeno también se entiende por la forma de la distribución. En posiciones donde un carácter concentra la mayor probabilidad, greedy descarta todas las alternativas plausibles. Una desviación estocástica podría sacar al modelo del patrón, pero greedy nunca la toma.

b. Comparación de B y C

La razón entre dos probabilidades después de aplicar temperatura es

$$\frac{P_i}{P_j} = \exp\left(\frac{z_i - z_j}{\tau}\right). \quad (5)$$

Con $\tau = 0.7$, B amplifica las diferencias entre logits; además, $k = 5$ restringe el soporte a cinco caracteres. Por ello produce palabras más estables y fragmentos reconocibles, como “All the prince” en B1. En C, $\tau = 1.0$ conserva la distribución aprendida y $k = 50$ permite casi todo el vocabulario.

La diversidad sube de 0.716 a 0.883, pero aparecen formas deformadas como “begainst”, “famour” y “Maughter’s” en C2.

Según estas muestras, B es más coherente y C es más creativo. Como k y τ cambian simultáneamente, no es posible atribuir toda la diferencia a uno solo de los parámetros.

c. Adaptación de top-p

La muestra E1 permite observar cómo cambia el núcleo. Los momentos con mayor seguridad ocurren al completar patrones ortográficos casi determinados:

Cuadro 4: Momentos de mayor seguridad en E1.

Paso	Contexto	Carácter	Candidatos	Entropía
139	MENENIUS:	salto de línea	1	0.002820
137	MENENIU	S	1	0.003057
138	MENENIUS	:	1	0.003065
136	MENENI	U	1	0.003289

Los momentos de mayor incertidumbre aparecen principalmente al comenzar una palabra, cuando muchas continuaciones son posibles:

Cuadro 5: Momentos de mayor incertidumbre en E1.

Paso	Contexto	Carácter	Candidatos	Entropía
16	thy heart!	F	32	3.292192
37	my lord of	i	32	3.068105
44	d of itself + salto	A	30	2.926484
34	ll, my lord	o	27	3.169611

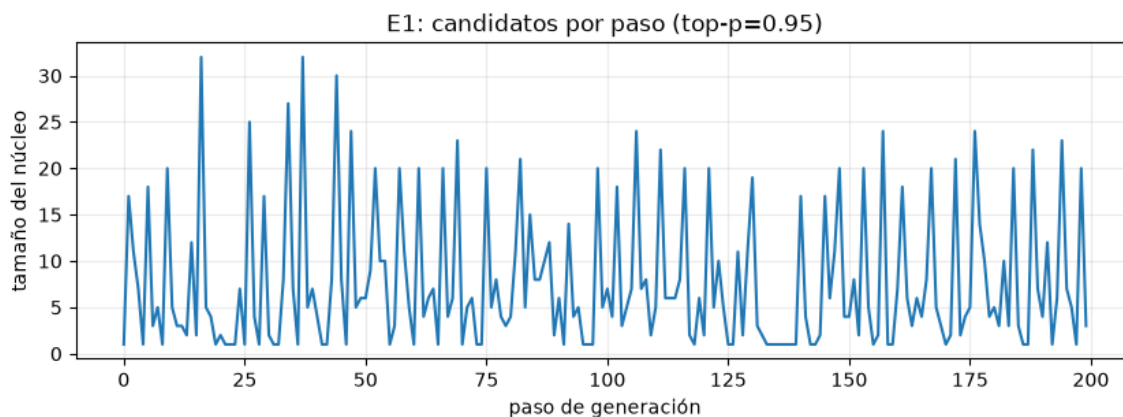


Figura 2: Tamaño del núcleo durante la generación de E1.

La Figura 2 confirma que top-p usa pocos candidatos dentro de secuencias predecibles y amplía el soporte en posiciones ambiguas. Top-k no puede realizar esta adaptación porque mantiene un valor fijo en todos los pasos.

3. Task 3 - Embeddings contextuales

3.1. Task 3.1 - Extracción con BERT

Se cargó `bert-base-multilingual-cased` y se localizaron las palabras objetivo mediante los offsets del tokenizador. En los siete casos la palabra completa correspondió a un solo token. Se tomó su representación de la última capa, no el vector de [CLS]. Cada embedding tiene dimensión 768.

Cuadro 6: Oraciones y palabras objetivo.

Oración	Palabra
El banco central anunció una subida de tasas de interés.	banco
Me senté en el banco del parque a leer el periódico.	banco
El banco de peces se movió velozmente ante el tiburón.	banco
El equipo levantó la copa del torneo ante miles de aficionados.	copa
Sirvió una copa de vino tinto para acompañar la cena.	copa
El velero desplegó su vela para aprovechar el viento.	vela
Encendió una vela para iluminar el cuarto durante el apagón.	vela

3.2. Task 3.2 - Similitud coseno y Word2Vec

La similitud entre dos embeddings se calculó mediante

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}. \quad (6)$$

Para la comparación estática se utilizó el modelo Word2Vec CBOW de 300 dimensiones del Spanish Billion Word Corpus. Word2Vec asigna un único vector a cada palabra, por lo que dos apariciones de la misma palabra tienen similitud 1, sin importar el contexto.

Cuadro 7: Comparación entre similitudes contextuales y estáticas.

Palabra	Contexto 1	Contexto 2	BERT	Word2Vec
banco	finanzas	asiento	0.5753	1.0000
banco	finanzas	peces	0.6603	1.0000
banco	asiento	peces	0.6496	1.0000
copa	trofeo	bebida	0.5667	1.0000
vela	navegación	cera	0.7999	1.0000

BERT produce valores menores que 1 en todos los pares, lo que demuestra que el vector cambia con la oración. La separación es más marcada para *copa*, con 0.5667, y más moderada para *vela*, con 0.7999. Las tres acepciones de *banco* quedan entre 0.5753 y 0.6603.

3.3. Reducción PCA

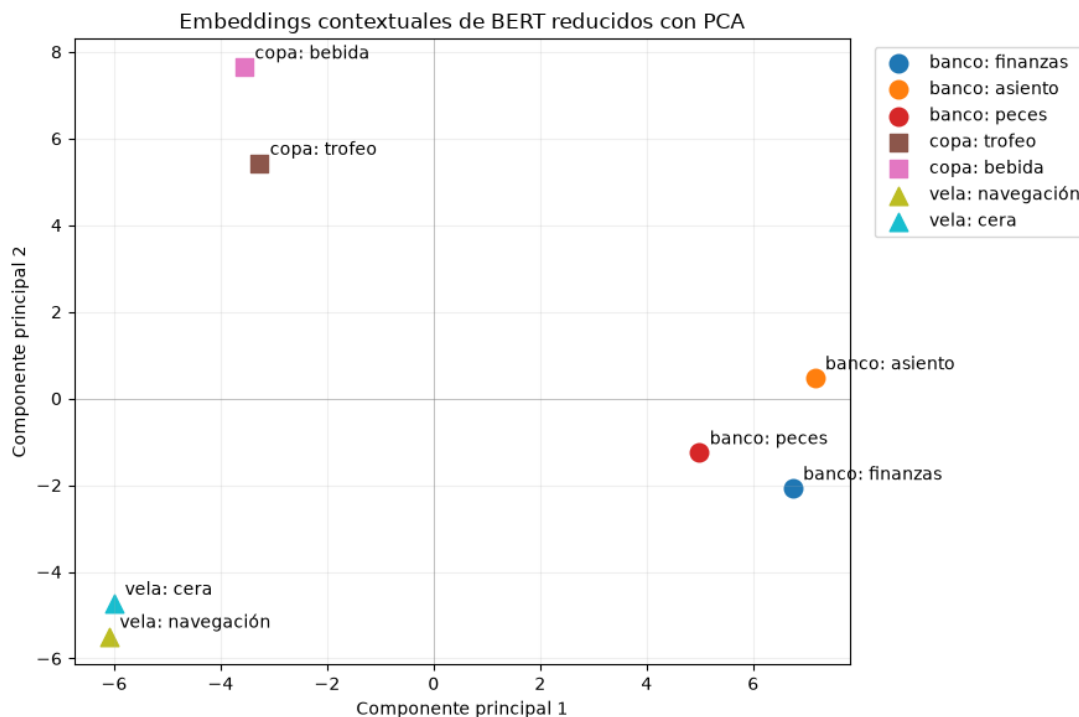


Figura 3: Embeddings de BERT proyectados a dos dimensiones mediante PCA.

La primera componente explica 35.02% de la varianza y la segunda 23.54%, para un total de 58.56%. En la Figura 3, la identidad de la palabra explica una parte importante de la disposición global: *banco*, *copa* y *vela* forman regiones diferentes. Dentro de cada palabra, cada significado ocupa una posición distinta, aunque la reducción a dos dimensiones comprime parte de la información presente en los 768 componentes. Por ello, la tabla de similitudes constituye la evidencia cuantitativa principal.

4. Task 4 - Discusión conceptual

4.1. Task 4.1(a) - Necesidad de la máscara causal

El objetivo autoregresivo factoriza la probabilidad de una secuencia como

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_{<t}). \quad (7)$$

La máscara causal garantiza que la representación utilizada para predecir x_{t+1} dependa únicamente de $x_{\leq t}$. Si GPT tuviera atención bidireccional durante este entrenamiento, la posición t podría consultar x_{t+1} , que es precisamente el token objetivo. La pérdida bajaría por fuga de información y dejaría de medir la capacidad de predecir el siguiente token. Durante la generación el futuro no está disponible, por lo que también existiría una discrepancia entre entrenamiento e inferencia.

4.2. Task 4.1(b) - Elección entre GPT y BERT

BERT usa contexto izquierdo y derecho durante el preentrenamiento enmascarado. Esta atención bidireccional es adecuada para representar una oración completa, pero no define directamente una distribución autoregresiva del siguiente token. Los resultados del Task 3 muestran su ventaja contextual: Word2Vec mantiene similitud 1 entre dos apariciones de la misma palabra, mientras BERT produce representaciones diferentes para cada oración.

GPT también produce representaciones dependientes del contexto, pero al generar solo puede utilizar el prefijo disponible. Su arquitectura causal está alineada con la continuación de texto. El paradigma de preentrenar y adaptar permite escoger el modelo según la tarea: GPT para generación y BERT para clasificación, extracción o comparación semántica con contexto bidireccional. Ambos aprovechan conocimiento lingüístico adquirido en grandes corpus y evitan entrenar un modelo desde cero.

5. Conclusiones

El Mini-GPT alcanzó una perplejidad adecuada y aprendió estructura local del texto teatral. Greedy produjo secuencias deterministas y repetitivas; top-k permitió controlar directamente el tamaño del soporte; y top-p ajustó el número de candidatos según la incertidumbre local. Los parámetros más restrictivos favorecieron coherencia, mientras que configuraciones amplias elevaron la diversidad a costa de errores ortográficos y sintácticos.

La comparación de embeddings confirmó la diferencia entre representaciones estáticas y contextuales. Word2Vec no distingue sentidos porque entrega el mismo vector en cada aparición. BERT sí modifica el embedding de acuerdo con la oración, lo que se reflejó tanto en las similitudes coseno como en la proyección PCA.

Fuentes y herramientas

- Dataset *tiny_shakespeare*: <https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt>
- BERT multilingüe: <https://huggingface.co/bert-base-multilingual-cased>
- Spanish Billion Word Corpus and Embeddings: <https://crscardellino.github.io/SBWCE/>

Se utilizó asistencia automatizada para contrastar el notebook con la consigna y localizar errores técnicos. Resumen del prompt utilizado: “Analiza el laboratorio y el trabajo existente, y dime si hay algún error grave para corregir”. Fue útil porque fijó el alcance de la revisión y exigió comprobar cada resultado mediante la ejecución del código.